

BIOMETRIA - PROJEKT 2

Rozpoznawanie człowieka na podstawie obrazu tęczówki

Zuzanna Zalewska, Ada Wojterska

Kwiecień 2026

1 Cel zadania

Celem projektu było opracowanie systemu rozpoznawania człowieka na podstawie obrazu tęczówki oka. Zadanie polegało na implementacji algorytmu segmentacji tęczówki, który umożliwia jej wyodrębnienie z obrazu oraz przekształcenie do postaci umożliwiającej dalszą analizę i porównywanie pomiędzy różnymi próbkami.

Zakres projektu obejmował dwa główne etapy.

W pierwszej części zaimplementowano algorytmy przetwarzania obrazu, które pozwoliły wyodrębnić tęczówkę. W tym celu obraz został przekształcony do skali szarości, a następnie poddany binaryzacji. W celu poprawy jakości segmentacji zastosowano operacje morfologiczne, które umożliwiły usunięcie szumów oraz wypełnienie ubytków. Kolejnym krokiem była detekcja granic źrenicy oraz tęczówki, a następnie rozwinięcie tęczówki do postaci prostokątnej.

W drugim etapie projektu rozwinięty obraz tęczówki do postaci prostokątnej został przekształcony do postaci kodu tęczówki. W tym celu obraz podzielono na pasy radialne, które następnie przekształcono do sygnałów jednowymiarowych. Na tak przygotowanych sygnałach zastosowano filtrację Gabora. Wyniki filtracji zostały zakodowane do postaci binarnej, tworząc kod tęczówki.

Końcowym elementem drugiego etapu było porównywanie kodów tęczówek przy użyciu odległości Hamminga, która określa stopień podobieństwa pomiędzy dwiema próbkami. Dzięki temu możliwe było rozstrzygnięcie, czy dwa obrazy należą do tej samej osoby.

2 Wykorzystane narzędzia

Projekt został zrealizowany w języku Python z wykorzystaniem następujących bibliotek:

- Tkinter – do stworzenia interfejsu graficznego
- NumPy – do operacji numerycznych oraz przetwarzania obrazów w postaci macierzy
- OpenCV – wykorzystana pomocniczo, m.in. do rysowania wykrytych okręgów
- Pillow – do wczytywania, konwersji oraz wyświetlania obrazów

3 Architektura systemu

Struktura projektu obejmuje następujące pliki:

- **main.py** – główny plik uruchamiający aplikację.
- **gui.py** – zawiera klasę odpowiedzialną za interfejs użytkownika.
- **image_ops.py** – zawiera podstawowe operacje przetwarzania obrazu, takie jak konwersja do skali szarości, binaryzacja oraz operacje morfologiczne.
- **projections.py** – odpowiada za analizę obrazu binarnego z wykorzystaniem projekcji. Zawiera funkcje wyznaczające środek oraz promień źrenicy i tęczy.
- **iris_engine.py** – zawiera główną logikę przetwarzania tęczy, w tym zastosowanie operacji morfologicznych oraz implementację rozwinięcia tęczy do postaci prostokątnej.
- **daugman.py** – odpowiada za implementację funkcji niezbędnych do wyznaczenia kodu tęczy oraz obliczania odległości Hamminga.
- **testowe_obrazy/** – katalog zawierający przykładowe obrazy wykorzystywane do testowania działania aplikacji.

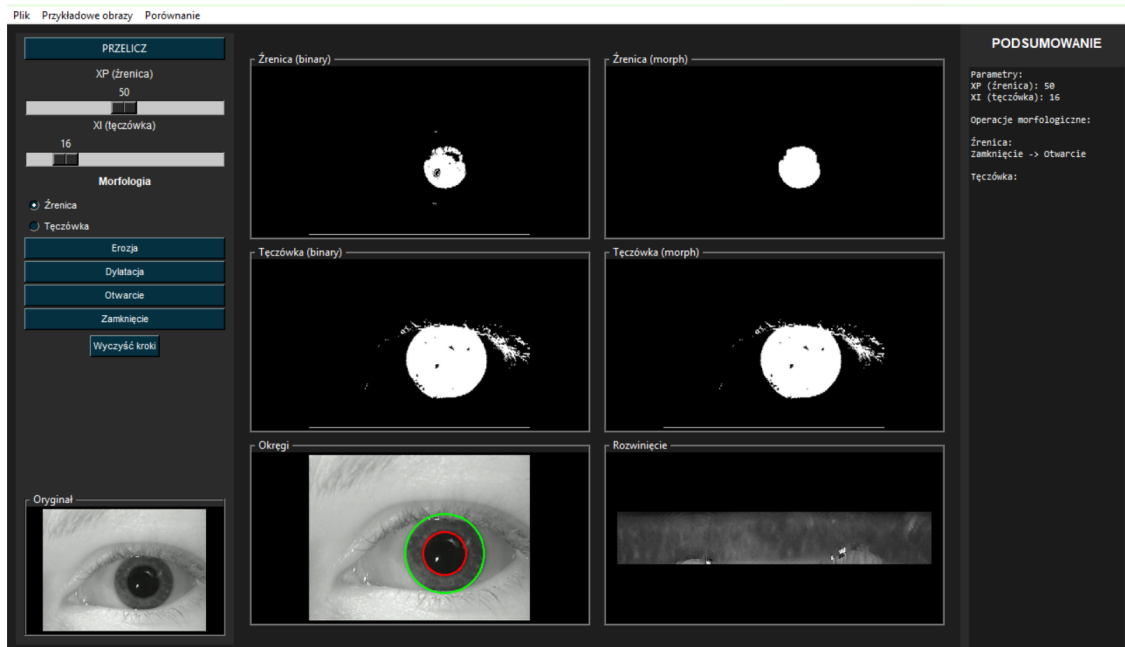
4 Opis interfejsu użytkownika

Interfejs użytkownika został podzielony na dwa tryby: tryb analizy oraz tryb porównania.

W trybie analizy interfejs składa się z trzech głównych paneli. W lewym panelu znajdują się elementy sterujące działaniem algorytmu. Użytkownik może regulować parametry X_P oraz X_I za pomocą suwaków, co pozwala na dostosowanie progów binaryzacji dla źrenicy i tęczy. Panel ten umożliwia również wybór operacji morfologicznych, takich jak erozja, dylatacja, otwarcie i zamknięcie, a także określenie, czy mają być one zastosowane do źrenicy czy tęczy. Dodatkowo dostępna jest funkcja ponownego przeliczenia wyników po zmianie parametrów.

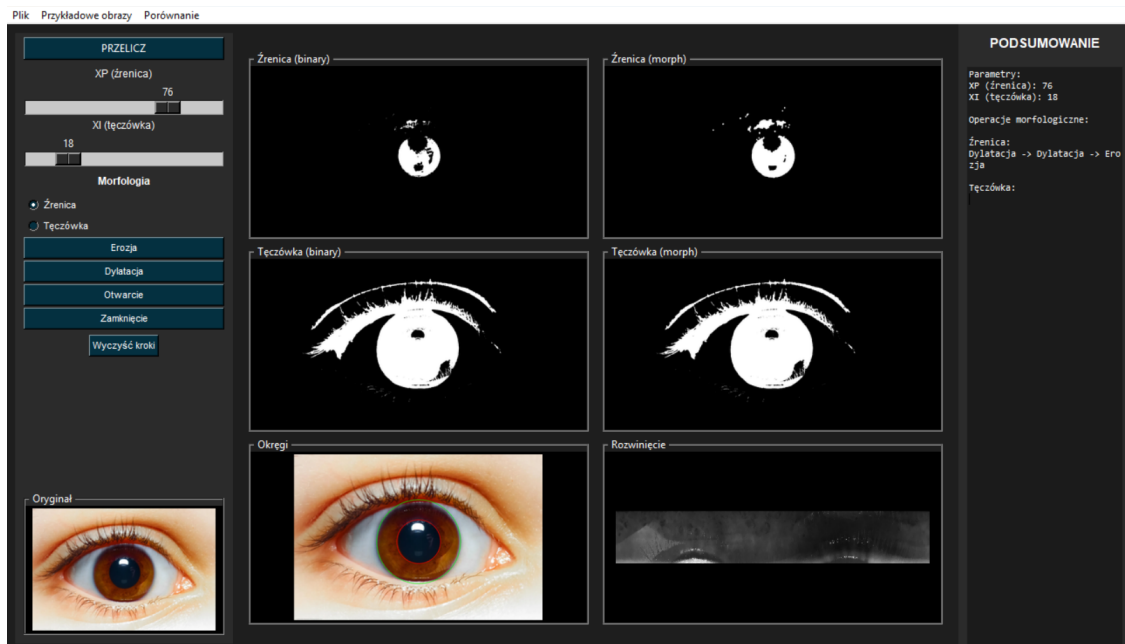
Środkowy panel służy do wizualizacji kolejnych etapów przetwarzania obrazu. Wyświetlane są obrazy binarne źrenicy i tęczy, wyniki po operacjach morfologicznych, obraz z zaznaczonymi granicami w postaci okręgów oraz rozwinięta tęć w postaci prostokątnej. Umożliwia to bieżącą analizę wpływu poszczególnych operacji na jakość segmentacji.

Prawy panel zawiera podsumowanie aktualnych ustawień, w tym wartości parametrów oraz listę zastosowanych operacji morfologicznych. Wygląd trybu analizy przedstawiono na rysunku 1.



Rysunek 1: Widok trybu analizy

Ponadto w górnej części interfejsu znajduje się pasek menu umożliwiający dostęp do dodatkowych funkcji aplikacji. Zakładka „Plik” pozwala na wczytanie własnego obrazu oraz zamknięcie aplikacji. Przykład działania systemu dla kolorowego wczytanego przez użytkownika obrazu przedstawiono na rysunku 2.



Rysunek 2: Przykład działania systemu dla obrazu wczytanego przez użytkownika

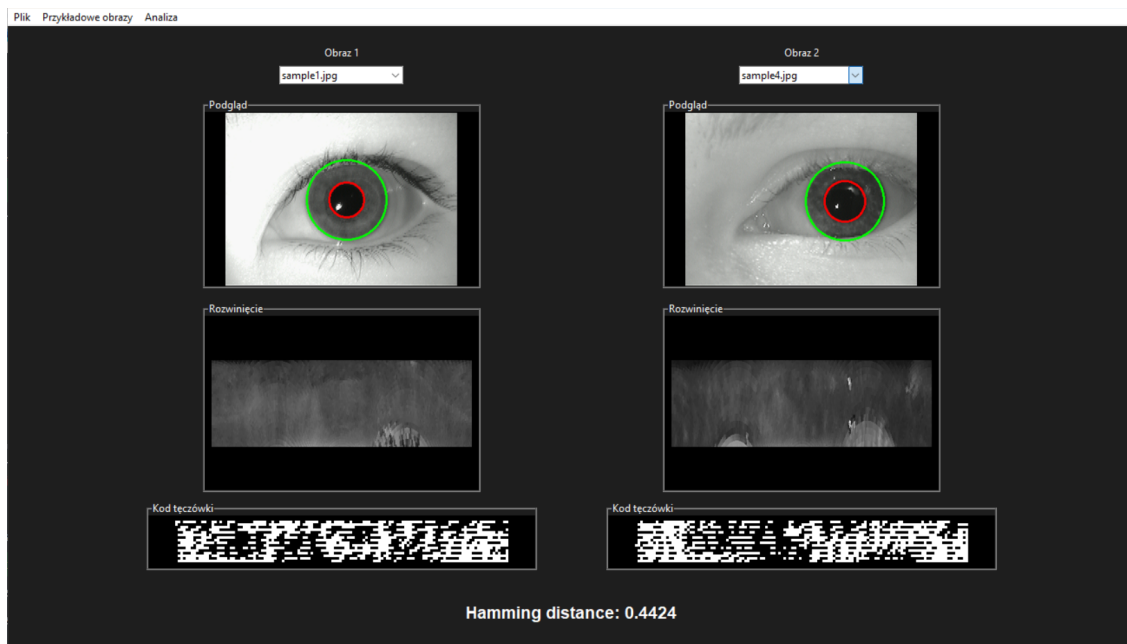
W zakładce „Przykładowe obrazy” użytkownik ma możliwość wyboru jednego z pięciu przygotowanych obrazów testowych, dla których domyślnie ustawione są odpowiednie

wartości parametrów X_P i X_I , a także wstępnie dobrane operacje morfologiczne dla źrenicy, co pozwala na szybkie rozpoczęcie analizy.

Dostępna jest również zakładka „Porównanie”, która umożliwia przejście do trybu porównywania dwóch obrazów tęczówki. W tym trybie interfejs podzielony jest na dwie kolumny, odpowiadające wybranym obrazom z listy przykładowych próbek.

W każdej kolumnie prezentowane są trzy wizualizacje ułożone jedna pod drugą. Pierwsza przedstawia obraz z zaznaczonymi okręgami, druga pokazuje rozwiniętą tęczówkę w postaci prostokątnej, natomiast trzecia przedstawia wygenerowany kod tęczówki.

Pod wizualizacjami wyświetlana jest wartość odległości Hamminga, która określa stopień podobieństwa pomiędzy porównywanymi obrazami. Wygląd trybu porównania przedstawiono na rysunku 3.



Rysunek 3: Widok trybu porównania

5 Etap 1

5.1 Przekształcenie do skali szarości

Obraz wejściowy w formacie RGB został przekształcony do skali szarości przy użyciu wzoru luminancji:

$$I = 0.299R + 0.587G + 0.114B \quad (1)$$

Implementacja została wykonana poprzez iterację po wszystkich pikselach obrazu i obliczenie wartości jasności dla każdego z nich.

5.2 Binarystacja

Binarystacja obrazu polegała na przekształceniu obrazu w skali szarości do postaci binarnej. Próg binarystacji wyznaczany był na podstawie średniej jasności obrazu:

$$P = \frac{1}{h \cdot w} \sum_{i=0}^{h-1} \sum_{j=0}^{w-1} A(i, j) \quad (2)$$

gdzie: h – wysokość obrazu, w – szerokość obrazu, $A(i, j)$ – jasność piksela w skali szarości na pozycji (i, j)

Następnie wyznaczano dwa progi:

- dla źrenicy: $P_P = \frac{P}{X_P}$,
- dla tęczówki: $P_I = \frac{P}{X_I}$.

Parametry X_P oraz X_I były dobierane eksperymentalnie za pomocą suwaków na interfejsie graficznym. Suwaki przyjmują wartości z zakresu od 1 do 100, przy czym w obliczeniach wartości te są dzielone przez 10 (czyli rzeczywisty możliwy zakres parametrów to $[0,1 - 10]$).

Dla przykładowych obrazów zastosowano wstępnie dobrane wartości parametrów, które zapewniają jak najlepszą segmentację, jednak użytkownik ma możliwość ich ręcznej modyfikacji w razie potrzeby.

5.3 Operacje morfologiczne

W celu poprawy jakości segmentacji zastosowano operacje morfologiczne: erozję, dyatację, otwarcie oraz zamknięcie.

Operacje te były wykonywane na obrazie binarnym w celu usunięcia szumów, wypełnienia ubytków w obszarze źrenicy (np. wynikających z odbłasków) oraz eliminacji niepożądanych elementów, takich jak rzęsy czy powieki.

W implementacji wykorzystano kod opracowany w ramach poprzedniego projektu. Operacje morfologiczne realizowane były z użyciem elementu strukturalnego o rozmiarze 7×7 .

5.4 Detekcja środka źrenicy

Środek źrenicy wyznaczany był na podstawie analizy rozmieszczenia białych pikseli w obrazie binarnym z wykorzystaniem projekcji pionowych i poziomych. Dodatkowo zastosowano projekcje pod różnymi kątami (0° , 30° , 60° , 90°), co zwiększyło dokładność detekcji.

Dla każdego kąta punkty były rzutowane na układ współrzędnych obrócony o dany kąt, a następnie wyznaczano przybliżone położenie środka jako medianę rzutów wzdłuż obu osi tego układu. Otrzymane w ten sposób przybliżenia środka były następnie uśredniane, co pozwalało uzyskać wynik odporny na zakłócenia i szum.

5.5 Wyznaczanie promienia

Promień źrenicy oraz tęczówki wyznaczany był poprzez analizę punktów na okręgach o rosnącym promieniu. Dla każdego promienia obliczano stosunek pikseli białych do wszystkich punktów na okręgu.

Jeżeli stosunek ten spadał poniżej określonego progu - w naszym przypadku był to eksperymentalnie wyznaczony próg 0.4, uznawano, że osiągnięto granicę źrenicy lub tęczówki.

5.6 Rozwinięcie tęczówki

Następnie wyznaczony obraz tęczówki został przekształcony do postaci prostokątnej. Proces ten polega na odwzorowaniu pikseli z układu biegunowego na układ kartezjański, co pozwala na „rozprostowanie” struktury tęczówki do formy umożliwiającej dalszą analizę.

W pierwszym kroku dla każdego punktu w obrazie wynikowym wyznaczany jest odpowiadający mu kąt θ , który zmienia się w zakresie od 0 do 2π . Następnie dla każdej wartości kąta wyznaczany jest promień r , który również zmienia się od granicy źrenicy do zewnętrznej granicy tęczówki.

Na podstawie tych wartości wyznaczane są współrzędne w obrazie wejściowym, z których pobierana jest wartość piksela i przypisywana do obrazu wynikowego. W efekcie otrzymywany jest prostokątny obraz tęczówki, w którym oś pozioma odpowiada zmianie kąta, a oś pionowa zmianie odległości od środka źrenicy.

6 Etap 2

6.1 Podział na pasy radialne

Pierwszym krokiem przetwarzania było przycięcie rozwiniętego obrazu tęczówki z górnej oraz dolnej części o wartość 0.1 wysokości obrazu. Zabieg ten miał na celu eliminację niepożądanych elementów, takich jak fragmenty źrenicy oraz rzęsy, które mogłyby negatywnie wpływać na dalszą analizę. Następnie przetworzony obraz został podzielony na osiem równych pasów radialnych.

6.2 Przekształcenie do reprezentacji jednowymiarowej

Każdy z wyodrębnionych pasów radialnych został przekształcony do postaci jednowymiarowej. W tym celu każdy pas podzielono na 128 równomiernie rozmieszczonych segmentów.

W każdym z segmentów obliczano średnią intensywność pikseli. Zastosowano ważenie wartości z wykorzystaniem okna Gaussa. Dzięki temu większe znaczenie przypisywano pikselom znajdującym się bliżej centralnej części analizowanego obszaru. Dodatkowo wykonano uśrednianie poziomu jasności w kierunku radialnym.

W wyniku opisanych operacji każdy pas został opisany przez 128 wartości liczbowych, które są następnie wykorzystywane w dalszym etapie przetwarzania.

6.3 Zastosowanie falek Gabora

Na przekształconej reprezentacji każdego pasa radialnego zastosowano analizę z wykorzystaniem falek Gabora.

W implementacji zastosowano domyślną wartość częstotliwości filtra wynoszącą $f = \frac{1}{4}$. Dla każdego fragmentu obrazu obliczana jest odpowiedź filtra Gabora, na podstawie której wyznaczone są składowe rzeczywiste i urojone. Następnie wartości te są przekształcane do postaci binarnej, co pozwala na utworzenie kodu tęczy.

6.4 Obliczanie odległości Hamminga

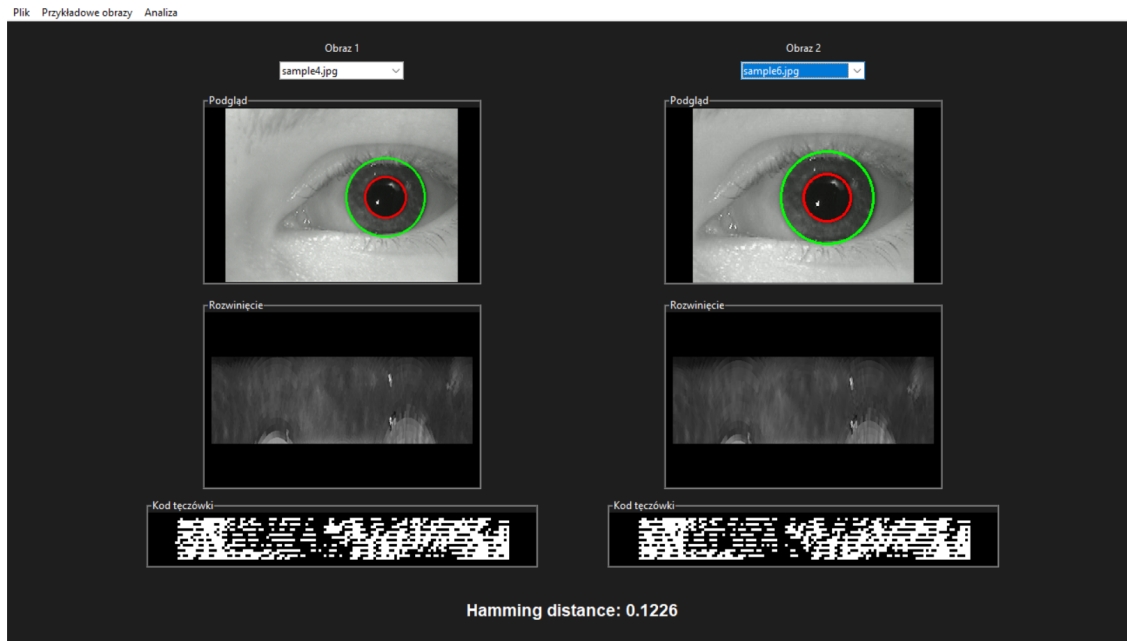
Ostatnim etapem jest porównanie uzyskanych kodów tęczy. W tym celu wykorzystano odległość Hamminga. Odległość ta obliczana jest jako średnia liczba pozycji, w których odpowiadające sobie bity dwóch kodów różnią się od siebie.

7 Wnioski

Zaimplementowany system pozwala na skuteczną segmentację tęczy oraz jej przekształcenie do postaci umożliwiającej dalszą analizę.

Największe trudności sprawił nam dobór odpowiednich progów binaryzacji oraz operacji morfologicznych, które pozwalały uzyskać obraz nadający się do dalszej analizy. Istotnym problemem była także eliminacja zakłóceń, takich jak rzęsy i powieki, które znacząco utrudniały pracę z obrazem. Dodatkowo wyzwaniem okazało się dla nas znalezienie sposobu na dokładne wyznaczenie promienia tęczy. W szczególności nieprecyzyjne określenie granic źrenicy i tęczy prowadziło do zniekształceń w etapie rozwinięcia obrazu.

Pomimo tych ograniczeń, zastosowane podejście pozwoliło na uzyskanie rozróżnialnych kodów tęczy, gdzie wartości odległości Hamminga dla różnych oczu oscylowały w zakresie 0.4–0.5 (co przedstawiono na rysunku 3). Zgodnie z intuicją, dla identycznych obrazów wartość ta wynosiła 0, natomiast dla różnych zdjęć tego samego oka przyjmowała wartości z zakresu 0.10–0.25. Na rysunku 4 przedstawiono przykład takiej sytuacji.



Rysunek 4: Porównanie różnych zdjęć tego samego oka

Literatura

- [1] Janusz Rafałko. Materiały do rozpoznawania tęczówki dostępne na stronie. https://pages.mini.pw.edu.pl/~rafalkoj/www/?Dydaktyka:2025%2F2026:-_Biometria,2026.
- [2] John Daugman. How iris recognition works. <https://www.robots.ox.ac.uk/~az/lectures/est/iris.pdf>, 2002.
- [3] A. J. Dixit and K. S. Kazi. Iris recognition by daugman's method. *International Journal of Latest Technology in Engineering, Management & Applied Science (IJLTEMAS)*, 2015. https://www.researchgate.net/publication/362569513_IRIS_Recognition_by_Daugmans_Method.
- [4] Mateusz Poleski. Iris recognition system. <https://github.com/mateuszpoleski/iris-recognition>, 2022. Simple iris recognition system, implemented with Python and OpenCV.
- [5] Naureen Mohammad. Mmu iris dataset. <https://www.kaggle.com/datasets/naureenmohammad/mmu-iris-dataset>, 2020. Zbiór danych zawierający obrazy tęczówki oka wykorzystane w projekcie.